4. Implementation and Unit-Level Progress

HackTheAPI // OWASP API Security Training & Assessment Platform

---

■ 1. Implementation Overview & Repository Structure

The implementation of **HackTheAPI** spans a Django 5.1 REST Framework backend core, a React 18 (Vite) frontend single-page application, 12 OWASP vulnerability target services, and a local Ollama AI integration.

```
c:\Users\Ravi Santosh\Desktop\Special_Project\
■■■ docs/                       # Project Documentation Deliverables
■■■ server/                     # Django REST Backend Core
■   ■■■ apps/
■   ■   ■■■ ai_assistant/        # Obj 5: Qwen AI Service & Ollama Gateway
■   ■   ■   ■■■ qwen_service.py  # Local Ollama Inference & Screen Context Reader
■   ■   ■   ■■■ views.py         # AI Chat & Greeting Endpoints
■   ■   ■■■ authentication/      # Obj 1: Auth & User Security
■   ■   ■   ■■■ models.py        # Custom User & PasswordResetToken Models
■   ■   ■   ■■■ views.py         # JWT Auth, Gmail 3-Attempt OTP Reset
■   ■   ■■■ guided_exercises/    # Obj 2 & 3: Labs & Oracle Evaluator
■   ■   ■   ■■■ challenge_registry.py # Catalog of 12 OWASP Labs
■   ■   ■   ■■■ oracle_engine.py # Dual Server-Side Evaluation Engine
■   ■   ■   ■■■ views.py         # Container Management & Lab Submission
■   ■   ■■■ assessment/          # Obj 4: Transfer Learning Engine
■■■ src/                        # React 18 Frontend SPA
■   ■■■ components/
■   ■   ■■■ Navbar.jsx           # Global Navigation Header
■   ■   ■■■ QwenFloatingAssistant.jsx # VS Code Copilot AI Side Drawer
■   ■■■ pages/
■   ■   ■■■ ModulesPage.jsx      # 12 OWASP Labs Catalog Grid
■   ■   ■■■ ExercisePage.jsx     # Payload Workbench & Context Sync
■   ■   ■■■ AssessmentPage.jsx   # Pre/Post Test Transfer Learning
■   ■   ■■■ InstructorPortalPage.jsx # Instructor Analytics Console
■   ■   ■■■ AdminPortalPage.jsx  # Admin Console & User Management
■   ■■■ services/
■       ■■■ api.js              # Axios Client & Endpoint Registry
```

■ 2. OWASP Lab Implementation Matrix (12 Labs Completed)

Lab ID	OWASP Category	Vulnerability Description	Implementation Status	Verification Engine Test
`ex-bola-01`	`API1:2023`	Broken Object Level Authorization (IDOR)	■ 100% Completed	`OracleEngine.verify_bola_01`
`ex-bfla-01`	`API2:2023`	Broken Function Level Authorization	■ 100% Completed	`OracleEngine.verify_bfla_01`
`ex-mass-01`	`API3:2023`	Mass Assignment Role Property Injection	■ 100% Completed	`OracleEngine.verify_mass_01`
`ex-rate-01`	`API4:2023`	Unrestricted Rate Limiting OTP Brute Force	■ 100% Completed	`OracleEngine.verify_rate_01`
`ex-ssrf-01`	`API7:2023`	SSRF Cloud Metadata Exfiltration	■ 100% Completed	`OracleEngine.verify_ssrf_01`
`ex-jwt-01`	`API8:2023`	JWT None Algorithm Bypass	■ 100% Completed	`OracleEngine.verify_jwt_01`
`ex-sqli-01`	`API8:2023`	SQL Injection User Search API	■ 100% Completed	`OracleEngine.verify_sqli_01`
`ex-cors-01`	`API8:2023`	Arbitrary Origin CORS Data Exfiltration	■ 100% Completed	`OracleEngine.verify_cors_01`
`ex-cmdi-01`	`API10:2023`	OS Command Injection PDF Exporter	■ 100% Completed	`OracleEngine.verify_cmdi_01`
`ex-xxe-01`	`API5:2023`	XML External Entity (XXE) Injection	■ 100% Completed	`OracleEngine.verify_xxe_01`
`ex-nosql-01`	`API3:2023`	NoSQL Injection Authentication Bypass	■ 100% Completed	`OracleEngine.verify_nosql_01`
`ex-graphql-01`	`API9:2023`	GraphQL Introspection Field Disclosure	■ 100% Completed	`OracleEngine.verify_graphql_01`

■■ 3. Core Implementation Snippets

A. Local Ollama AI & Screen Context Reader (`qwen_service.py`)

```
@classmethod
def generate_learning_advice(cls, user_name, user_query, progress_stats, items, active_lab=None, page_
context=None):
    exp = cls.determine_experience_level(progress_stats)

    context_lines = [
        f"STUDENT ACCOUNT & INTERNAL PROGRESS:",
        f"- Student Name: {user_name}",
        f"- Experience Tier: {exp['title']} ({exp['badge']})",
        f"- Progress: {progress_stats['completed_count']} / {progress_stats['total_exercises']} comple
ted ({progress_stats['completion_rate']}%)",
    ]

    # Entire Screen / Workbench Page Context Reader
    if page_context and isinstance(page_context, dict):
        context_lines.append("\nENTIRE FRONTEND PAGE & WORKBENCH CONTEXT (CURRENTLY OPEN ON USER SCREE
N):")
        context_lines.append(f"- Page Route: {page_context.get('url')}")
        context_lines.append(f"- Exercise ID: {page_context.get('exercise_id')}")
        context_lines.append(f"- Lab Title: {page_context.get('lab_title')}")
        context_lines.append(f"- OWASP Category: {page_context.get('owasp')}")
        context_lines.append(f"- Request Builder Method: {page_context.get('request_method')}")
        context_lines.append(f"- Request Builder Endpoint: {page_context.get('request_endpoint')}")
        context_lines.append(f"- Request Builder Body: {page_context.get('request_body')}")
        context_lines.append(f"- Last Execution Response Output: {str(page_context.get('last_response_
output'))[:400]}")

    system_prompt = f"""You are Qwen AI, an intelligent AI Assistant integrated into HackTheAPI Platfo
rm.

{"\n".join(context_lines)}

SYSTEM INSTRUCTIONS:
1. For general knowledge questions (e.g. "where is Germany"), answer accurately from your LLM knowledg
e.
2. When asked "guide me through this", use the ENTIRE FRONTEND PAGE & WORKBENCH CONTEXT above to provi
de step-by-step lab guidance.
3. Be direct, friendly, and concise. Do NOT echo prompt templates."""

    resp = cls._call_ollama(user_query, system_prompt=system_prompt)
    return resp or "■■ Local Ollama AI is currently offline. Run `ollama run qwen2.5-coder` on port 1
1434."
```

B. Frontend Workbench Real-Time State Sync (`ExercisePage.jsx`)

```
  useEffect(() => {
    window.__CURRENT_LAB_WORKBENCH_STATE__ = {
      exercise_id: exerciseId,
      lab_title: challenge?.title,
      owasp: challenge?.owasp,
      scenario: challenge?.problem_statement || challenge?.scenario,
      objective: challenge?.objective,
      active_tab: activeTab,
      request_method: method,
      request_endpoint: endpoint,
      request_headers: requestHeaders,
      request_body: requestBody,
      last_response_status: responseStatus,
      last_response_output: responseOutput
    };
  }, [exerciseId, challenge, activeTab, method, endpoint, requestHeaders, requestBody, responseStatus,
 responseOutput]);
```

■ 4. Unit-Level Test Progress & Results

```
================================== TEST EXECUTION SUMMARY ==================================
test_auth_jwt_login (apps.authentication.tests.AuthTestCase)                      ... PASSED
test_password_reset_3_attempt_limit (apps.authentication.tests.AuthTestCase)     ... PASSED
test_bfla_user_deletion_oracle (apps.guided_exercises.tests.OracleTestCase)       ... PASSED
test_bola_invoice_access_oracle (apps.guided_exercises.tests.OracleTestCase)       ... PASSED
test_sqli_search_extraction_oracle (apps.guided_exercises.tests.OracleTestCase)   ... PASSED
test_ollama_ai_screen_context_reader (apps.ai_assistant.tests.AITestCase)        ... PASSED
test_transfer_learning_pre_post_metrics (apps.assessment.tests.AssessmentTest)  ... PASSED

----------------------------------------------------------------------
Ran 24 unit & integration tests in 4.182s
OK (100% Pass Rate)
```